Atomically stores a new value and reads the old one.

Declaration

```
BaseType exchange(
    BaseType new_value,
    memory_order order = memory_order_seq_cst)
    volatile noexcept;
```

Effects

Atomically stores `new_value` in `*this` and retrieves the existing value of `*this`.

Returns

The value of `*this` immediately prior to the store.

Throws

Nothing.

Note

This is an atomic read-modify-write operation for the memory location comprising `*this`.

Std::Atomic_Exchange Nonmember Function

Atomically stores a new value in an `atomic<BaseType>` instance and reads the prior value.

Declaration

```
template<typename BaseType>
BaseType atomic_exchange(volatile atomic<BaseType>* p, BaseType
```